ACTIVITY 2- IMPLEMENTATION OF STACK ADT

Name: B. Siva Parvathi
Reg no:192525194
Course Code: CSA0312
Course Name: Data Structures

1) To implement the using: Array Linked List
The program should demonstrate the basic stack operations using the principle.

Part A: Stack Implementation Using Array
Write a menu-driven C program to implement a stack using an array Operations to Implement Push an element Pop an element Peek the top element Display stack elements Check whether the stack is empty Check whether the stack is full
Exit Sample Menu STACK USING ARRAY
1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Check Full
7. Exit

Enter your choice: Conditions to Handle

- Display when the stack is full.
- Display when the stack is empty.
- Display the top element using the Peek operation.
- Display stack elements from top to bottom.

Programiz
C Online Compiler

Programiz PRO >

main.c

```c
#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;
void push() {
    int x;
    if (top == MAX - 1)
        printf("Stack is Full\n");
    else {
        printf("Enter element: ");
        scanf("%d", &x);
        stack[++top] = x;
    }
}
void pop() {
    if (top == -1)
        printf("Stack is Empty\n");
    else
        printf("Deleted element: %d\n", stack[top--]);
}
void peek() {
    if (top == -1)
        printf("Stack is Empty\n");
```

Output

```
STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 1
Enter element: 10

STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 1
Enter element: 20
```

Programiz
C Online Compiler

Programiz PRO >

main.c

```c
        printf("Stack is Empty\n");
    else
        printf("Top element: %d\n", stack[top]);
}
void display() {
    int i;
    if (top == -1)
        printf("Stack is Empty\n");
    else {
        printf("Stack elements:\n");
        for (i = top; i >= 0; i--)
            printf("%d ", stack[i]);
        printf("\n");
    }
}
void checkEmpty() {
    if (top == -1)
        printf("Stack is Empty\n");
    else
        printf("Stack is Not Empty\n");
}
void checkFull() {
```

Output

```
STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 1
Enter element: 30

STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 4
Stack elements:
30 20 10
```

Programiz
C Online Compiler
Programiz PRO >

main.c Run

```c
        printf("Stack is Not Empty\n");
}
void checkFull() {
    if (top == MAX - 1)
        printf("Stack is Full\n");
    else
        printf("Stack is Not Full\n");
}
int main() {
    int ch;
    do {
        printf("\nSTACK USING ARRAY\n");
        printf("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Check
            Empty\n6.Check Full\n7.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: checkEmpty(); break;
```

Output Clear

```
STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 4
Stack elements:
30 20 10

STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 3
Top element: 30
```

Online C Compiler - Programiz

Online C Compiler - Programiz

Programiz
C Online Compiler
Programiz PRO >

main.c Run

```c
    do {
        printf("\nSTACK USING ARRAY\n");
        printf("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Check
            Empty\n6.Check Full\n7.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: checkEmpty(); break;
            case 6: checkFull(); break;
            case 7: printf("Exit\n"); break;
            default: printf("Invalid Choice\n");
        }
    } while (ch != 7);
    return 0;
}
```

Output Clear

```
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 3
Top element: 30

STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 2
Deleted element: 30
```

Online C Compiler - Programiz

Programiz C Online Compiler

Programiz PRO

main.c

Run

```c
        printf("Stack is Not Full\n");
}
int main() {
    int ch;
    do {
        printf("\nSTACK USING ARRAY\n");
        printf("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Check
            Empty\n6.Check Full\n7.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: checkEmpty(); break;
            case 6: checkFull(); break;
            case 7: printf("Exit\n"); break;
            default: printf("Invalid Choice\n");
        }
    } while (ch != 7);
    return 0;
}
```

Output

Clear

```
STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 4
Stack elements:
20 10

STACK USING ARRAY
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Check Full
7.Exit
Enter your choice: 5
Stack is Not Empty
```

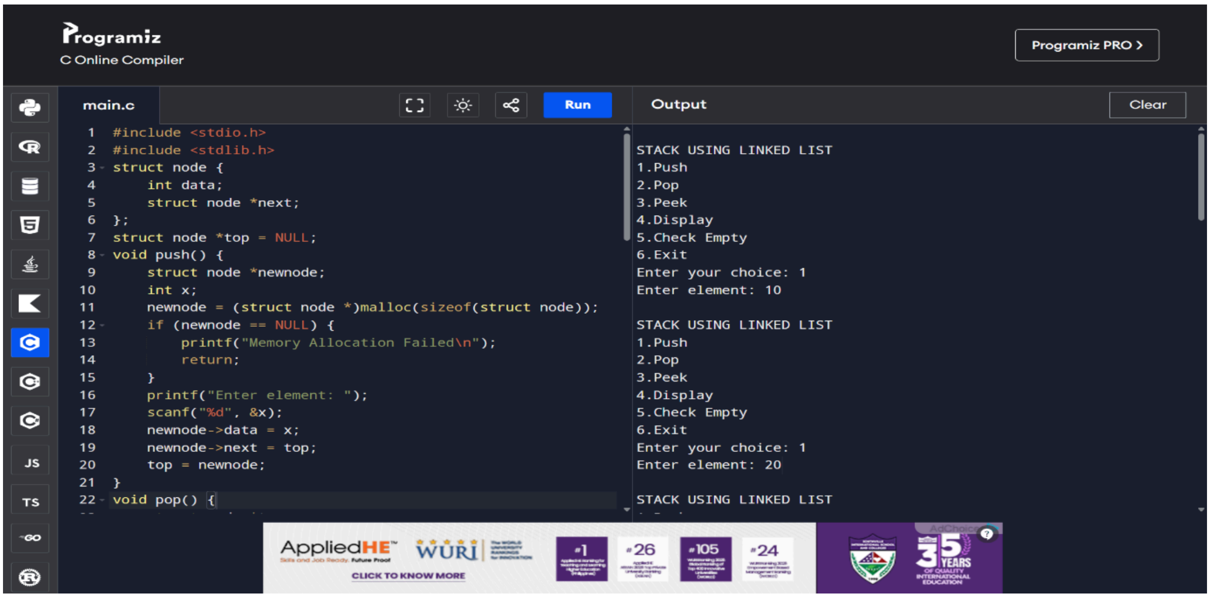
Part B: Stack Implementation Using Linked List

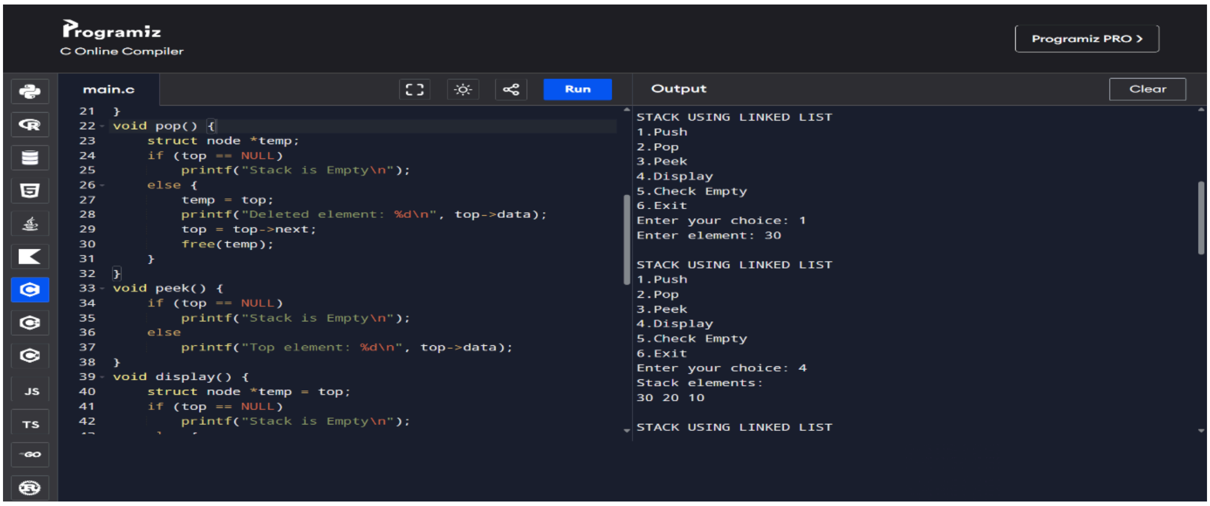
Write a menu-driven C program to implement a stack using a linked list Operations to Implement Push an element Pop an element Peek the top element Display stack elements Check whether the stack is empty Exit Sample Menu STACK USING LINKED LIST

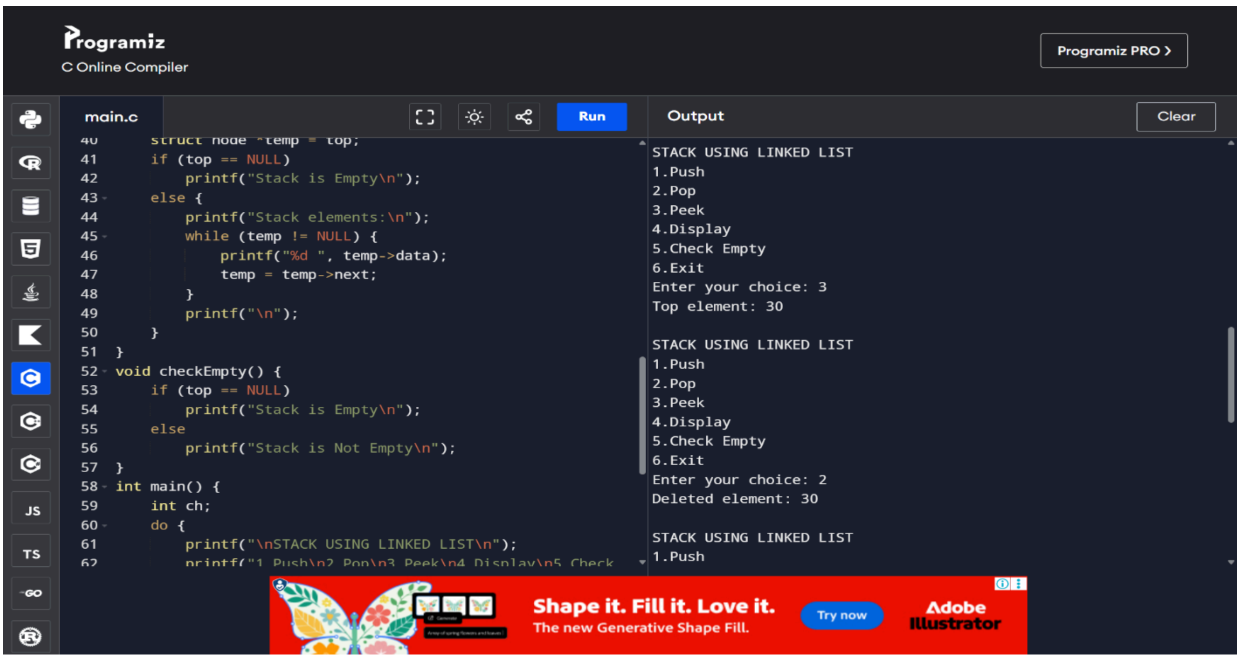
1. Push
2. Pop
3. Peek
4. Display
5. Check Empty
6. Exit

Enter your choice: Conditions to Handle

- Create a new node dynamically during the Push operation.
- Delete the top node during the Pop operation.
- Display when the stack is empty.
- Display the elements from top to bottom.
- Handle memory allocation failure.


Programiz
C Online Compiler
Programiz PRO >
main.c
Run
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};
struct node *top = NULL;
void push() {
    struct node *newnode;
    int x;
    newnode = (struct node *)malloc(sizeof(struct node));
    if (newnode == NULL) {
        printf("Memory Allocation Failed\n");
        return;
    }
    printf("Enter element: ");
    scanf("%d", &x);
    newnode->data = x;
    newnode->next = top;
    top = newnode;
}
void pop() {
Output
Clear
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 1
Enter element: 10
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 1
Enter element: 20
STACK USING LINKED LIST
AppliedHE
WURI
CLICK TO KNOW MORE
#1
#26
#105
#24
35 YEARS


Programiz
C Online Compiler
Programiz PRO >
main.c
Run
}
void pop() {
    struct node *temp;
    if (top == NULL)
        printf("Stack is Empty\n");
    else {
        temp = top;
        printf("Deleted element: %d\n", top->data);
        top = top->next;
        free(temp);
    }
}
void peek() {
    if (top == NULL)
        printf("Stack is Empty\n");
    else
        printf("Top element: %d\n", top->data);
}
void display() {
    struct node *temp = top;
    if (top == NULL)
        printf("Stack is Empty\n");
Output
Clear
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 1
Enter element: 30
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 4
Stack elements:
30 20 10
STACK USING LINKED LIST


Programiz
C Online Compiler
Programiz PRO >
main.c
Run
    if (top == NULL)
        printf("Stack is Empty\n");
    else {
        printf("Stack elements:\n");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}
void checkEmpty() {
    if (top == NULL)
        printf("Stack is Empty\n");
    else
        printf("Stack is Not Empty\n");
}
int main() {
    int ch;
    do {
        printf("\nSTACK USING LINKED LIST\n");
Output
Clear
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 3
Top element: 30
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 2
Deleted element: 30
STACK USING LINKED LIST
1.Push
Shape it. Fill it. Love it.
The new Generative Shape Fill.
Try now
Adobe Illustrator

Programiz
C Online Compiler

Programiz PRO >

main.c

Run

```c
        printf("Stack is Not Empty\n");
}
int main() {
    int ch;
    do {
        printf("\nSTACK USING LINKED LIST\n");
        printf("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Check Empty\n6.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: checkEmpty(); break;
            case 6: printf("Exit\n"); break;
            default: printf("Invalid Choice\n");
        }
    } while (ch != 6);
    return 0;
}
```

Output

Clear

```
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 4
Stack elements:
20 10

STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 5
Stack is Not Empty

STACK USING LINKED LIST
```


Programiz
C Online Compiler

Programiz PRO >

main.c

Run

```c
        printf("Stack is Not Empty\n");
}
int main() {
    int ch;
    do {
        printf("\nSTACK USING LINKED LIST\n");
        printf("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Check Empty\n6.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: checkEmpty(); break;
            case 6: printf("Exit\n"); break;
            default: printf("Invalid Choice\n");
        }
    } while (ch != 6);
    return 0;
}
```

Output

Clear

```
STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 5
Stack is Not Empty

STACK USING LINKED LIST
1.Push
2.Pop
3.Peek
4.Display
5.Check Empty
6.Exit
Enter your choice: 6
Exit

=== Code Execution Successful ===
```